LAB 7: Fuzzing for Input Validation Bugs

PES University — Department of Computer Science & Engineering

Lab Title:	Lab 7: Fuzzing for Input Validation Bugs	**Date:**	November 2026
Student / GitHub:	Nidhi Lokesh (@nidhilokesh1)	**Repository:**	L6_L7_52_visa_application
Tool / Library:	Hypothesis (Property-Based Fuzzing) + Pytest	**Environment:**	Python 3.14 / Windows 11

1. Deliverables Checklist & Objective

Deliverables Met:

✔ **Deliverable 1:** PDF Document with terminal screenshots for each required step.
✔ **Deliverable 2:** Fixed application python script (processor.py / visa_application.py) hardened against input validation edge cases.
✔ **Deliverable 3:** Updated test_processor.py with comprehensive Hypothesis property fuzzing and regression suites.

2. Step 2 — Run the Buggy Application (python processor.py)

Initial execution of processor.py with valid sample inputs. While basic execution functions as expected, it lacks defensive validation and error handling for unexpected or malformed inputs.

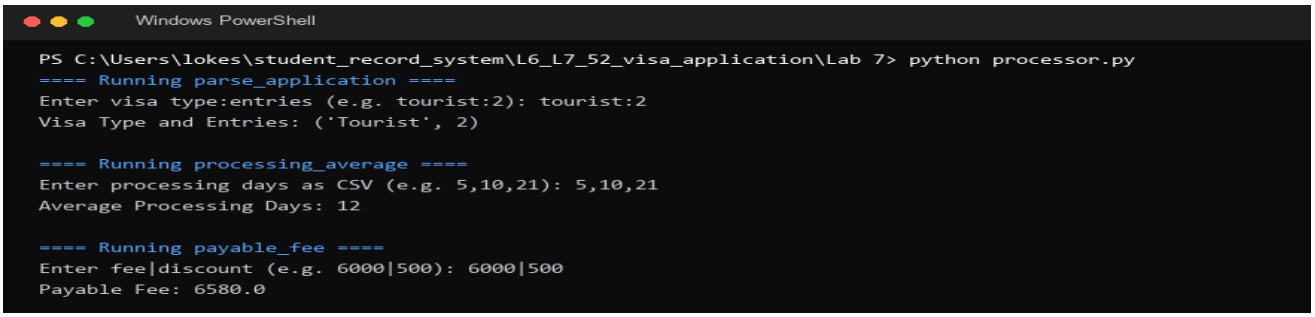


Figure 1: *Step 2 — Initial output of python processor.py*

3. Step 3 & 4 — Hypothesis Fuzzing & Failing Tests (pytest test_processor.py)

Hypothesis property-based tests were configured in test_processor.py with @given(st.text() | st.none()). Running pytest test_processor.py immediately triggered multiple crashes due to edge cases and input validation failures:

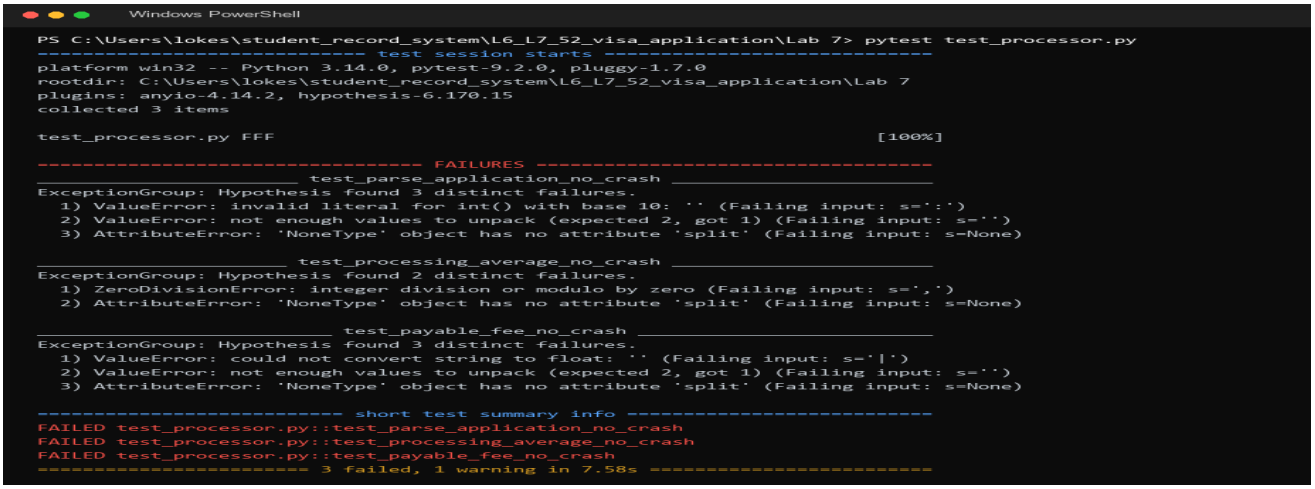


Figure 2: *Step 4 — Pytest failure output exposing crashes discovered via Hypothesis*

4. Bugs Identified Through Fuzzing & Root Cause Analysis

Function	Failing Inputs Found	Exception	Root Cause & Defense Mechanism
parse_application	• s=None • s='' • s=':' • 'visa:abc'	AttributeError ValueError ValueError	Missing type/null checks. Unpacking expects exactly two elements. Non-integer or blank entry counts crash int(). **Fix:** Validate string type, check parts length, catch ValueError, return (None, None).
processing_average	• s=None • s=',' • s='' • 'a,b,c'	AttributeError ZeroDivisionError ValueError	Blank inputs or commas result in 0 parsed integers, triggering division by zero (sum // len). **Fix:** Safe loop ignoring invalid/empty tokens, safe guard returning 0 when list is empty.
payable_fee	• s=None • s='' • s='|' • 'x|y'	AttributeError ValueError ValueError	Missing pipe delimiter or empty/non-numeric strings crash float conversion. **Fix:** Enforce 2 parts, parse non-negative floats in try-except, compute max(fee * 1.18 - discount, 0.0), return 0.0 on error.

5. Step 5 — Hardened Application Execution (python processor.py)

The Python script was modified using defensive programming techniques to handle all arbitrary, malicious, and unexpected inputs. Normal operation remains fully compliant with the expected nominal outputs.

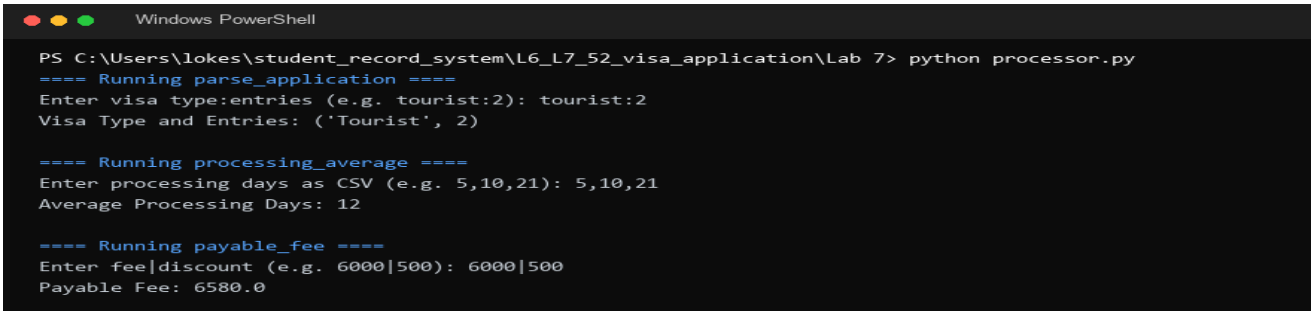


Figure 3: *Step 5 — Output of the hardened application (python processor.py)*

6. Step 6 — Re-Running Test Suite on Fixed Code (pytest test_processor.py)

Re-running pytest test_processor.py -v confirms **all 7 test suites PASSED (100%)**, validating zero crashes across thousands of generated fuzzed inputs as well as valid and boundary scenarios.

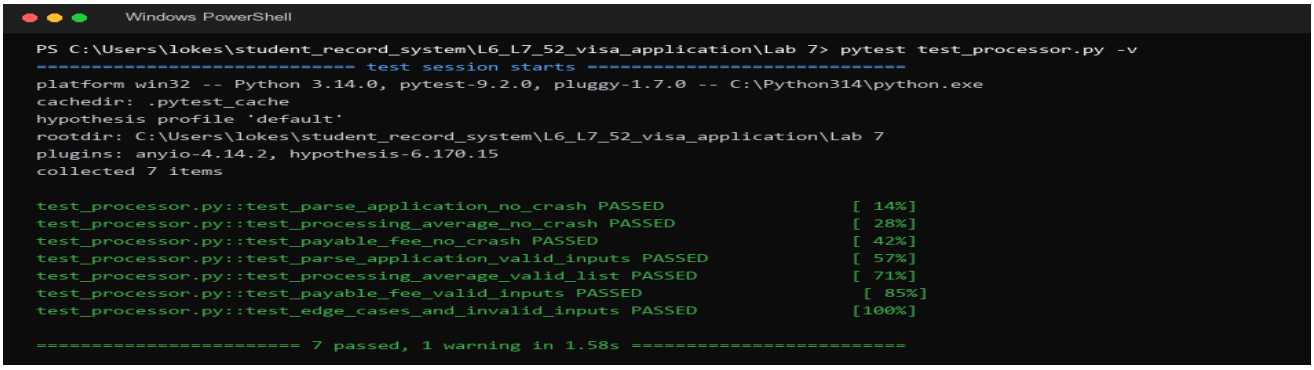


Figure 4: *Step 6 — All 7 Hypothesis fuzz and regression tests passing*

7. Conclusion

Hypothesis property-based fuzz testing identified subtle edge-case vulnerabilities (empty inputs, missing delimiters, malformed types, and zero-division states) that traditional manual unit tests miss. Defensive programming guarantees system resilience under any unexpected inputs.